

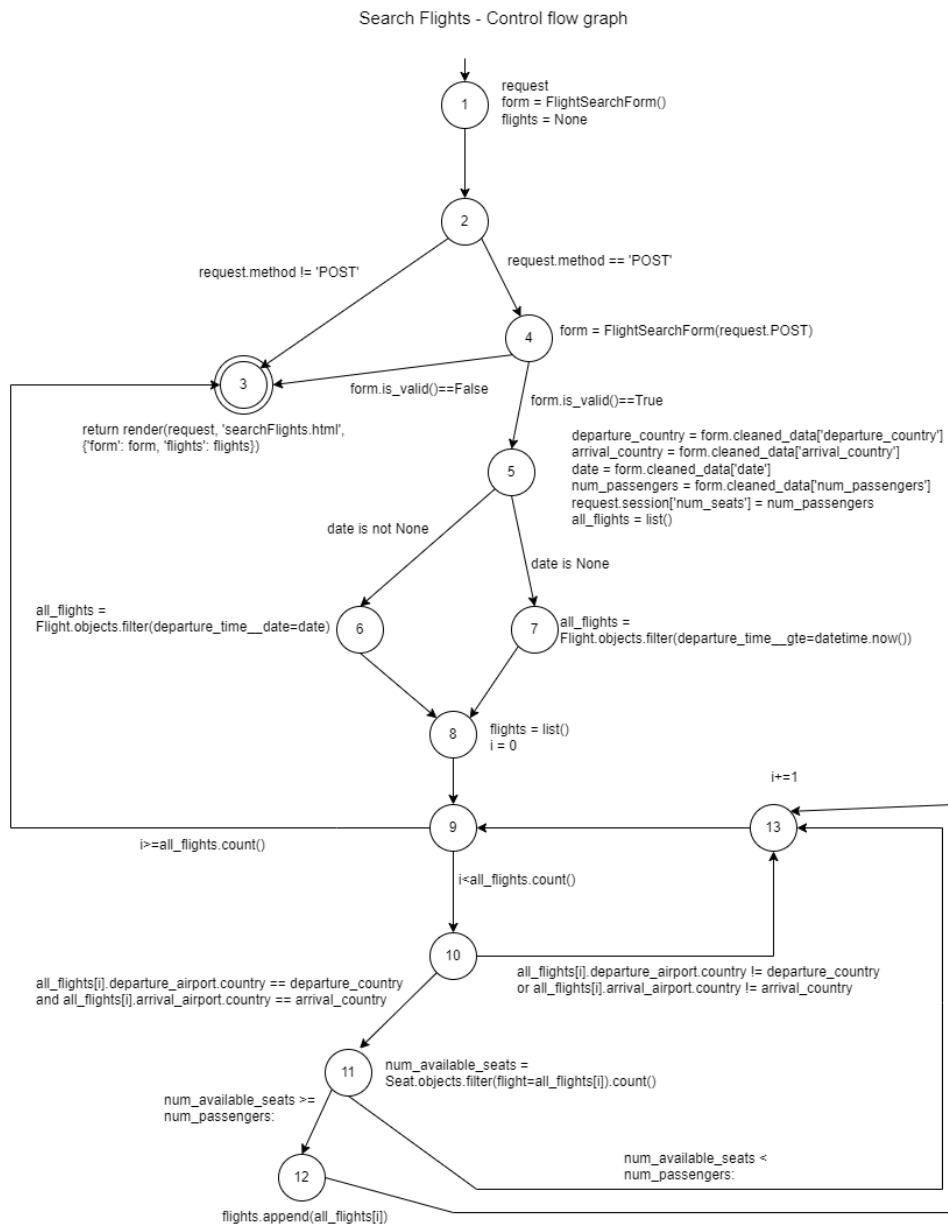
Testing of FlightTickets – app for booking flight tickets from various airlines

Jonatan Mileski

211512

Documentation

1. The app tested is a Django application, and in the scope of this project unit and E2E tests will be provided. For unit tests I will use Pytest, while for E2E tests I will use Cypress. Wherever applicable in the code, I used graph coverage and Edge-pair as a test criteria, and input space partitioning. Namely, for the functions in views.py tests are obtained using these coverage methods.
2. Unit Tests
 - a. Testing of searchFlights(request) method:



The image shown depicts the control graph of the function. Using Edge-pair coverage, the following test paths define the minimum test set:

[1,2,4,5,7,8,9,10,11,12,13,9,10,13,9,10,11,13,9,3]

[1,2,4,5,6,8,9,3]

[1,2,4,3]

[1,2,3]

For each of the paths there is one test, and the path is denoted above the test function in the code.

For Input Space Partitioning (ISP in the rest of the document):

Interface-based characteristics:

C1: Request method: C1.1 -> POST, C1.2 -> GET

C2: Form is valid: C2.1 -> True, C2.2-> False

Functionality-based characteristics:

C3: Number of flights which satisfy criteria: C3.1 -> 0, C3.2 -> 1, C3.3 -> >1

Test cases using BCC:

C1.1 C2.1 C3.2 – happy path

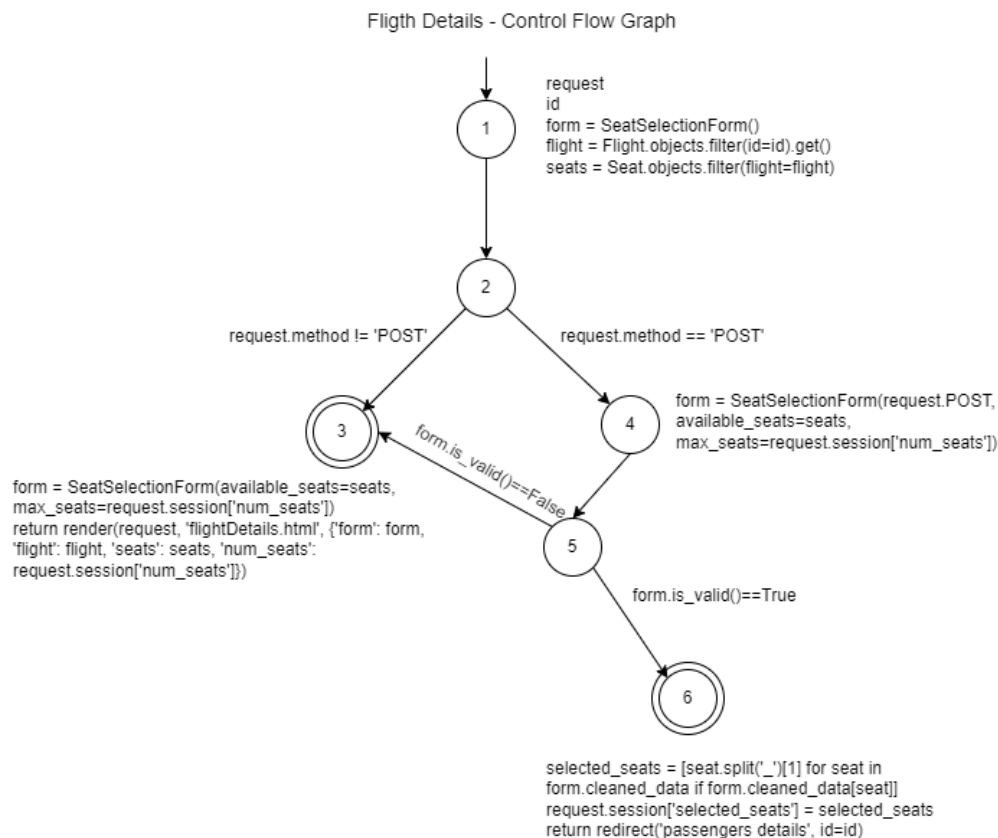
~~C1.2 C2.1 C3.2~~ infeasible

~~C1.1 C2.2 C3.2~~ infeasible

C1.1 C2.1 C3.1

C1.1 C2.1 C3.3

b. Testing of FlightDetails(request, id) method:



Using Edge-pair coverage, the following tests are obtained:

[1, 2, 4, 5, 6]

[1, 2, 4, 5, 3]

[1, 2, 3]

ISP:

Interface based characteristics:

C1: Request method: C1.1 -> POST, C1.2 -> GET

C2: Form is valid: C2.1 -> True, C2.2-> False

C3: Number of selected seats: C3.1 -> ≥ 1 , C3.2 -> 0

Functionality based:

C4: Redirection: C4.1 -> Passenger Details, C4.2-> Flight Details

Test cases using BCC:

C1.1 C2.1 C3.1 C4.1-happy path

~~C1.2 C2.1 C3.1 C4.1~~ infeasible

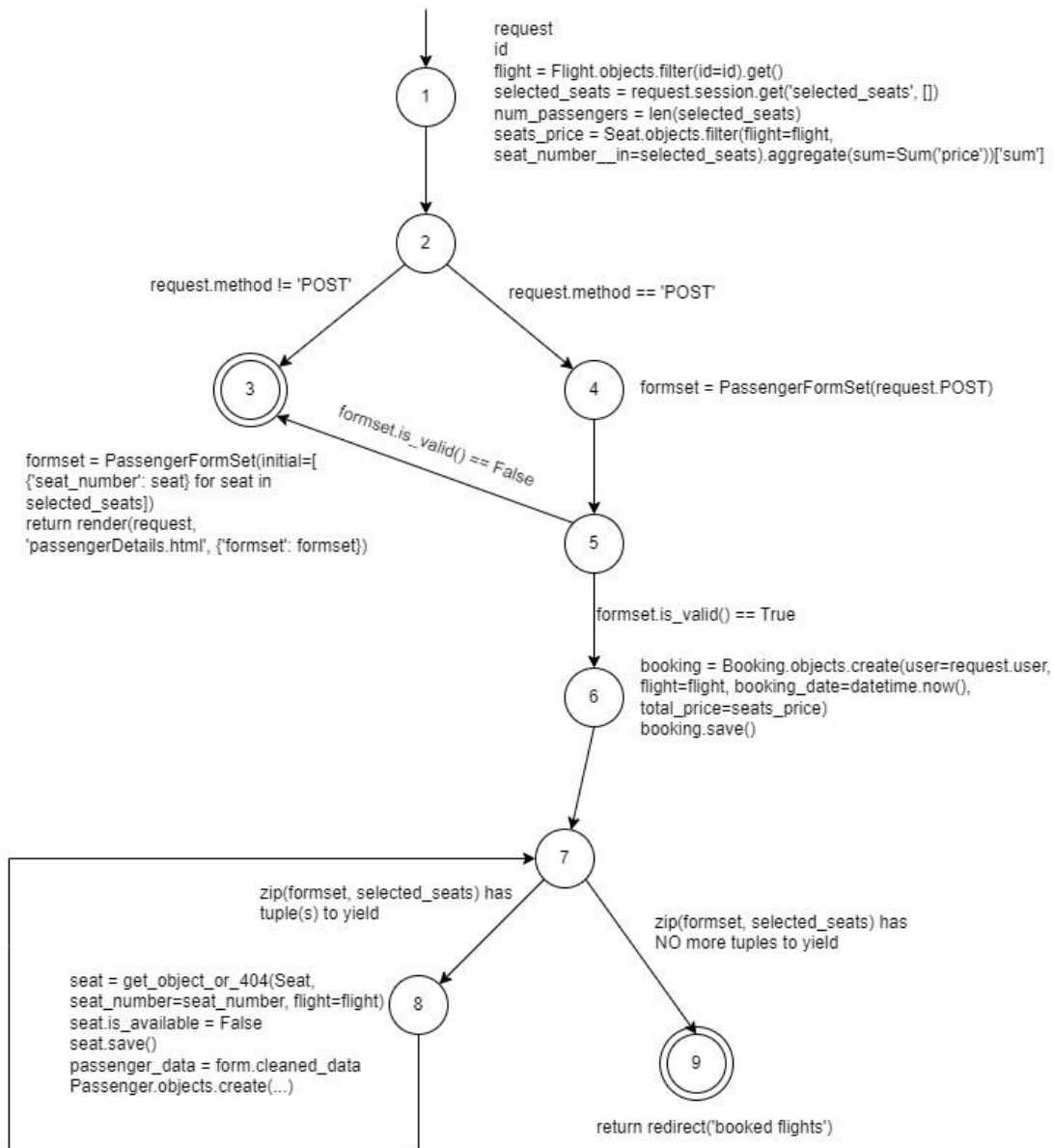
~~C1.1 C2.2 C3.1 C4.1~~ infeasible

~~C1.1 C2.1 C3.1 C4.2~~ infeasible

~~C1.1 C2.1 C3.2 C4.1~~ infeasible <- the method is not allowed to be executed if no seats are previously selected

c. Testing of passenger_details(request, id):

Passenger Details - Control Flow Graph



Obtained Edge-Pair coverage test set:

[1,2,4,5,6,7,8,7,8,7,9]

[1,2,4,5,6,7,9]

[1,2,4,5,3]

[1,2,3]

ISP:

Interface-based characteristics:

C1: Request method: C1.1 -> POST, C1.2 -> GET

C2: Form is valid: C2.1 -> True, C2.2-> False

Functionality-based:

C3. Redirection: C3.1-> Booked Flights, C3.2-> passenger details

Test cases using BCC:

C1.1 C2.1 C3.1 -happy path

~~C2.1 C2.1 C3.1~~

~~C1.1 C2.2 C3.1~~

~~C1.1 C2.1 C3.2~~

- d. Testing of `booked_flights(request)`:
Since this is a simpler function no graph coverage is needed.

ISP:

Interface-based:

C1: The user is logged in: C1.1 -> True C1.2 -> False

Functionality based:

C2: Number of passengers for which booking user is the logged in user: C2.1-> 0, C2.2-> >=1.

C1.1 C2.2– happy path

~~C1.2 C2.2~~

C1.1 C2.1

3. End-to-End Testing

For testing the end-to-end functionality, and the UI of the app, Cypress tests which encompass different possible scenarios of user flows are written. They include:

- No Filter Applied Search Flights
- Search Flights with Applied Filter
- Whole booking process
- Search Flights with Applied Filter and try to select an already booked seat or more than the specified number of seats, after which the whole procedure is completed after correcting the steps.
- Complete whole booking procedure, search again, logout, and try to see the booked flights

4. Running the tests:

- Unit tests

Run

`pytest`

in the terminal

- E2E tests

Run

`npx cypress open`

to open cypress

and then run:

```
python manage.py testserver TicketsApp/fixtures/test_data.json -  
-addrport 127.0.0.1:8000
```